

COS 102: PROBLEM SOLVING

Problems and Problem-Solving

A problem is a situation or question that requires a solution. It can be a challenge, obstacle, or difficulty that needs to be overcome to achieve a goal.

Problems are an inherent part of life, and problem-solving skills are essential for success in academics, careers, and personal life.

Types of Problems

1. **Routine Problems:** these are problems that are well-defined, have a clear solution path, and can often be solved using established procedures, formulas, or algorithms. Examples include:

- Mathematical calculations (e.g., solving an algebraic equation)
- Technical troubleshooting (e.g., fixing a computer error)
- Everyday tasks (e.g., cooking a meal using a recipe)

Approaches to routine problems:

- Following a step-by-step process
- Applying known formulas or rules
- Using tools or resources designed for the specific problem

2. **Non-Routine Problems:** these are poorly defined problems. They lack clear solution path and require creative thinking, critical analysis, and innovative solutions. Examples are:

- Designing a marketing campaign for a new product
- Resolving a complex ethical dilemma
- Writing a persuasive essay

Approaches to Non-Routine Problems

- Breaking down the problem into smaller, manageable parts
- Brainstorming and generating multiple potential solutions
- Evaluating and refining ideas
- Collaborating with others to gain diverse perspectives

The Problem-Solving Process

The process of problem solving include:

- i. **Identify the Problem:** Clearly define the issue or challenge you are facing.
- ii. **Gather Information:** Research and collect relevant data or knowledge.

- iii. **Generate Possible Solutions:** Brainstorm various approaches or options.
- iv. **Evaluate Solutions:** Analyze the pros and cons of each solution.
- v. **Choose the Best Solution:** Select the most effective and feasible option.
- vi. **Implement the Solution:** Put your chosen solution into action.
- vii. **Review the Results:** Assess the outcomes and make adjustments as needed.

Methods of Solving Computing Problems: Algorithms and Heuristics

Algorithms: this is a step-by-step procedure or set of rules for solving a specific problem or accomplishing a task.

Properties / Characteristics of Algorithm

- **Well-defined:** Each step is clear and unambiguous.
- **Finite:** The procedure terminates after a finite number of steps.
- **Correct:** Produces the correct output for valid input.
- **General:** Can be applied to a class of similar problems.

Examples of Algorithm

- Sorting algorithms (bubble sort, merge sort, quicksort)
- Searching algorithms (linear search, binary search)
- Graph algorithms (shortest path, minimum spanning tree)
- Numerical algorithms (finding roots of equations, numerical integration)

Applications of Algorithm

Algorithms are essential in various fields of computing, including software development, data processing, artificial intelligence, and scientific computing.

Heuristics: this involves a rule of thumb, shortcut, or educated guess that helps find a solution to a problem, but does not guarantee an optimal or correct solution.

Properties/Characteristics of Heuristics:

- **Approximate:** Provides a good enough solution in a reasonable amount of time.
- **Flexible:** Can be adapted to different problem variations.
- **Intuitive:** Often based on experience, intuition, or common sense.

Examples of Heuristics

- Greedy algorithms: Make locally optimal choices at each step, hoping for a global optimum.

- Hill climbing: Iteratively improve a solution by making small changes.
- Simulated annealing: Allow occasional "bad" moves to escape local optima.
- Genetic algorithms: Mimic natural selection to evolve solutions.

Applications of Heuristics

Heuristics are used when finding an optimal solution is computationally expensive or infeasible, or when the problem is ill-defined or complex.

Algorithms vs. Heuristics: A Comparison

Feature	Algorithms	Heuristics
Guarantee	Always produce a correct or optimal solution (if one exists)	May or may not produce the best solution
Computational Cost	Can be computationally expensive, especially for large or complex problems	Generally faster and less computationally demanding
Scope	Well-defined problems with clear objectives	Ill-defined or complex problems where finding the optimal solution is difficult
Optimality	Focus on finding the best possible solution	Focus on finding a good enough solution within reasonable time

Solvable and Unsolvable Problems

Solvable Problems

Solvable Problems are problems that have a definite solution or set of solutions that can be reached within a finite amount of time and with available resources.

Characteristics of Solvable Problems:

- **Well-defined:** The problem is clearly stated, with specific goals, constraints, and criteria for success.

- **Sufficient information:** There is enough information available to understand the problem and develop potential solutions.
- **Available resources:** The necessary tools, knowledge, and expertise are accessible to address the problem.
- **Feasibility:** The solution(s) can be implemented within practical limitations of time, cost, and resources.

Examples of Solvable problems include:

- Mathematical equations: Finding the roots of a quadratic equation.
- Scientific experiments: Determining the effect of a new drug on a specific disease.
- Engineering challenges: Designing a bridge that can withstand a certain load.
- Everyday problems: Planning a budget, organizing a schedule.

Unsolvable Problems

Unsolvable Problems are problems that have no solution or have solutions that are impossible to reach due to inherent limitations, contradictions, or lack of resources.

Characteristics Unsolvable Problems

- **Ill-defined or paradoxical:** The problem statement is vague, contradictory, or logically impossible.
- **Insufficient or contradictory information:** The available information is incomplete, unreliable, or conflicts with itself.
- **Lack of resources or expertise:** The necessary tools, knowledge, or skills are unavailable or insufficient.
- **Inherent constraints:** The problem is limited by physical laws, ethical principles, or social norms that cannot be overcome.

Examples of Unsolvable Problems include:

- Mathematical paradoxes: The "halting problem" in computer science (determining whether a program will halt or run forever).
- Philosophical questions: The meaning of life, the existence of free will.
- Social issues: Eliminating poverty, achieving world peace.
- Scientific limitations: Traveling faster than the speed of light, reversing the arrow of time.

Strategies for Dealing with Unsolvable Problems

- i. **Reframe the problem:** Try to redefine the problem in a way that makes it solvable or focus on more manageable aspect of the problem.
- ii. **Set realistic goals:** Instead of seeking a complete solution, aim for partial solutions or incremental improvements.
- iii. **Manage the consequences:** Develop strategies for coping with the negative effects of the unsolvable problem and mitigating its impact.
- iv. **Accept the limitations:** Recognize that some problems are beyond our current capabilities and learn to live with uncertainty or ambiguity.

Solution Techniques for Effective Problem Solving

Problem-Solving

Problem-solving is a fundamental skill for success in any field. It involves finding solutions to challenges, obstacles, or questions. There are numerous problem-solving techniques, each with its strengths and weaknesses.

Problem-Solving Techniques

1. **Abstraction:** Simplifying a complex problem by focusing on its essential features and ignoring irrelevant details. Example: Creating a diagram or model to represent a system.
2. **Analogy:** Finding similarities between a current problem and a previously solved problem, and applying the same solution strategy. Example: Using the analogy of a virus to explain the spread of computer malware.
3. **Brainstorming:** Generating a large number of potential solutions without judgment, then evaluating and refining them later. Example: Using a mind map to brainstorm ideas for a marketing campaign.
4. **Trial and Error:** Experimenting with different solutions until one works. Example: Trying different combinations of ingredients to create a new recipe.
5. **Hypothesis Testing:** Formulating a hypothesis about the cause of a problem and then testing it through experimentation or observation. Example: Conducting a survey to test a hypothesis about consumer preferences.
6. **Reduction:** Breaking down a complex problem into smaller, more manageable sub-problems. Example: Dividing a large research project into smaller tasks.

7. **Lateral Thinking:** Approaching a problem from a different, unexpected perspective to generate creative solutions. Example: Using wordplay or riddles to solve a puzzle.
8. **Means-End Analysis:** Identifying the goal and working backward to determine the steps needed to achieve it. Example: Planning a road trip by first deciding on the destination and then mapping out the route.
9. **Method of Focal Objects:** Combining characteristics of different objects to create a new, innovative solution. Example: Inventing a new product by combining features of existing products.
10. **Morphological Analysis:** Systematically exploring all possible combinations of parameters to identify potential solutions. Example: Designing a new type of vehicle by considering all possible combinations of engine, body style, and fuel type.
11. **Research:** Gathering information and knowledge relevant to the problem to inform the solution process. Example: Conducting a literature review before starting a research project.
12. **Root Cause Analysis:** Identifying the underlying cause of a problem, rather than just addressing its symptoms. Example: Investigating why a machine keeps breaking down, rather than just repairing it each time.
13. **Proof:** Using logical reasoning and evidence to demonstrate the validity of a solution. Example: Proving a mathematical theorem using axioms and previously proven theorems.
14. **Divide and Conquer:** Dividing a problem into smaller sub-problems, solving each sub-problem independently, and then combining the solutions. Example: Sorting a large list of numbers by first dividing it into smaller lists and then sorting each list individually.

General Problem-Solving Process and Solution Formulation /Design Using Flowcharts, Pseudocode, Decision Tables, And Decision Trees:

The General Problem-Solving Process include:

1. **Problem Definition:** this involves clearly stating of the problem, including the desired outcome and any constraints. Identify the inputs (known information) and the outputs (desired results). Ask clarifying questions to ensure a thorough understanding.
2. **Information Gathering:** involves research and collection of relevant data. Here you identify any assumptions or limitations and consider different perspectives and approaches.
3. **Solution Generation:** this has to do with brainstorming multiple potential solutions. The feasibility and effectiveness of each solution are considered here. It also involves breaking down of complex solutions into smaller steps.

4. **Solution Evaluation:** here we analyze the pros and cons of each solution. We also consider the resources needed and the potential impact. The most promising solution(s) is/are chosen.
5. **Solution Implementation:** involves development of a detailed plan for implementing the chosen solution. Test and refine the solution as needed. Also monitor the results and make adjustments if necessary.

Solution Formulation & Design Techniques

1. Flowcharts: Graphical representation of a process or algorithm, using standardized symbols for different actions (input/output, decisions, processes, etc.).

Advantages of flowchart

- Visual clarity
- Easy to follow,
- Useful for identifying logic errors.

Disadvantages of flowchart

- Can become complex for large problems,
- Not ideal for detailed data manipulation.

2. Pseudocode: it is an informal, high-level description of an algorithm, using a mix of natural language and programming constructs. It is an English like or natural language description of an algorithm.

Advantages of Pseudocode

- More expressive than flowcharts
- Focuses on logic without specific syntax
- Bridges the gap between human thought and code.

Disadvantages of Pseudocode

- Can be ambiguous
- Not directly executable.

Example: Write pseudocode for finding the average of two numbers.

3. Decision Tables: A decision table is a tabular representation of complex decision logic, showing different conditions and their corresponding actions.

Advantages of Decision Tables

- Clear and concise for representing multiple conditions
- Useful for testing and debugging.

Disadvantages of Decision Table

- Can be difficult to read for large tables
- Less intuitive than flowcharts or pseudocode for sequential processes.

4. Decision Trees: These are tree-like diagram showing decisions and their possible consequences.

Advantages Decision Trees

- Visual representation of hierarchical decision-making
- Helpful for visualizing probabilities and outcomes.

Disadvantages Decision Trees

- Can become unwieldy for complex problems
- May not capture all nuances of a situation.

Implementation, Evaluation, and Refinement, Specifically Focusing on Programming in C and Python,

Implementation: It is the process of turning a program design into actual code.

Implementation

Translating Algorithms to Code:

Steps for Translating an Algorithm into Program (Code)

Translating algorithm into a program involves five main steps:

- understanding the algorithm
- choosing a programming language suitable for the task,
- coding the algorithm in that language,
- testing to make sure it works properly and
- debugging to fix any issues. It is about knowing the problem, choosing the right tool and refining until you get the desired result.

Understanding the Algorithm: The first step in converting an algorithm into a program is to thoroughly understand the algorithm. This involves analyzing the problem the algorithm is designed to solve, understanding the logic behind the algorithm, and identifying the inputs and outputs. It is crucial to have a clear understanding of the algorithm's flow and the steps it takes to solve the problem. This understanding will guide the programming process and help in choosing the right programming language.

Choosing a Programming Language: The next step is to choose a suitable programming

language. The choice of language depends on several factors such as the nature of the problem, the platform on which the program will run, and the programmer's familiarity with the language. Some languages are better suited to certain types of problems than others. For example, Python is often used for data analysis and machine learning, while JavaScript is commonly used for web development.

Translating the algorithm into code: Once the programming language has been chosen, the next step is to translate the algorithm into code. This involves writing the instructions of the algorithm in the syntax of the chosen programming language. It is important to follow good coding practices, such as using meaningful variable names and adding comments to explain the code. This will make the code easier to understand and maintain.

Testing: After the code has been written, it needs to be tested to ensure it works as expected. This involves running the program with different inputs and checking the outputs. If the program does not produce the expected results, the code needs to be debugged.

Debugging: This involves identifying and fixing errors in the code. This can be a time-consuming process, but it is essential to ensure the program works correctly.

Evaluation: This is the process of testing and debugging code to ensure it works correctly.

Types of Testing:

- **Unit Testing:** Testing individual components or functions in isolation.
- **Integration Testing:** Testing how different components work together.
- **System Testing:** Testing the entire program as a whole.

Debugging Techniques:

- **Print Statements/Logging:** Inserting print statements (C) or using the logging module (Python) to track variable values and program flow.
- **Debuggers:** Introduce integrated development environments (IDEs) with debugging tools (e.g., breakpoints, step-through execution, variable inspection).
- **Code Review:** Having another person review your code to identify errors or potential issues.

Refinement (Optimization and Improvement): This is the process of optimizing and improving code for better performance (efficiency), readability, and maintainability.

- **Code Readability:**
 - **C:** to ensure readability in C programming, the following measures are taken:
 - i. Consistent indentation,
 - ii. meaningful variable names,
 - iii. Comments to explain complex logic.
 - **Python:** the measures taken to ensure code readability in Python, PEP 8 style guide for clean and readable code is followed.
- **Code Maintainability:** this is achieved through the following:
 - i. Modular design,
 - ii. clear documentation,
 - iii. well-structured code to make it easier to modify or fix in the future.
- **Code Efficiency:**
 - **C:** in C programming, the following helps in maintaining code efficiency:
 - i. Algorithm optimization
 - ii. efficient data structures,
 - iii. minimizing unnecessary computations
 - **Python:** to maintain code efficiency in Python, the following measures are taken:
 - i. Use built-in functions and libraries optimized for performance
 - ii. Avoid inefficient loops.
- **Profiling Tools:** Introduce profiling tools that help identify performance bottlenecks in the code.

Python programming Language

Python is a widely used programming language that offers several unique features and advantages compared to languages like **Java** and **C++**.

The first version of Python 0.9.0 was released BY Guido van Rossum in 1991. Since its release, Python started gaining popularity. According to reports, Python is now the most popular programming language among developers because of its high demands in the tech realm.

Features of python

- **Easy to use and Read** - Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers. This simplicity can lead to faster development and reduce the chances of errors.

- **Dynamically Typed** - The data types of variables are determined during run-time. We do not need to specify the data type of a variable during writing codes.
- **High-level** - High-level language means human readable code.
- **Compiled and Interpreted** - Python code first gets compiled into bytecode, and then interpreted line by line. When we download the Python in our system form **org** we download the default implement of Python known as CPython. CPython is considered to be Compiled and Interpreted both.
- **Garbage Collected** - Memory allocation and de-allocation are automatically managed. Programmers do not specifically need to manage the memory.
- **Purely Object-Oriented** - It refers to everything as an object, including numbers and strings.
- **Cross-platform Compatibility** - Python can be easily installed on Windows, macOS, and various Linux distributions, allowing developers to create software that runs across different operating systems.
- **Rich Standard Library** - Python comes with several standard libraries that provide ready-to-use modules and functions for various tasks, ranging from **web development** and **data manipulation** to **machine learning** and **networking**.
- **Open Source** - Python is an open-source, cost-free programming language. It is utilized in several sectors and disciplines as a result.

Areas of application of Python

Python is a general-purpose, popular programming language, and it is used in almost every technical field. The various areas of Python use are given below.

- **Data Science:** Data Science is a vast field, and Python is an important language for this field because of its simplicity, ease of use, and availability of powerful data analysis and visualization libraries like **NumPy**, **Pandas**, and **Matplotlib**.
- **Desktop Applications:** **PyQt** and **Tkinter** are useful libraries that can be used in GUI - Graphical User Interface-based Desktop Applications. There are better languages for this field, but it can be used with other languages for making Applications.
- **Console-based Applications:** Python is also commonly used to create command-line or console-based applications because of its ease of use and support for advanced features such as input/output redirection and piping.

- **Mobile Applications:** While Python is not commonly used for creating mobile applications, it can still be combined with frameworks like **Kivy** or **BeeWare** to create cross-platform mobile applications.
- **Software Development:** Python is considered one of the best software-making languages. Python is easily compatible with both from Small Scale to Large Scale software.
- **Artificial Intelligence:** AI is an emerging Technology, and Python is a perfect language for artificial intelligence and machine learning because of the availability of powerful libraries such as **TensorFlow**, **Keras**, and **PyTorch**.
- **Web Applications:** Python is commonly used in web development on the backend with frameworks like **Django** and **Flask** and on the front end with tools like **JavaScript HTML** and **CSS**.
- **Enterprise Applications:** Python can be used to develop large-scale enterprise applications with features such as distributed computing, networking, and parallel processing.
- **3D CAD Applications:** Python can be used for 3D computer-aided design (CAD) applications through libraries such as **Blender**.
- **Machine Learning:** Python is widely used for machine learning due to its simplicity, ease of use, and availability of powerful machine learning libraries.
- **Computer Vision or Image Processing Applications:** Python can be used for computer vision and image processing applications through powerful libraries such as **OpenCV** and **Scikit-image**.
- **Speech Recognition:** Python can be used for speech recognition applications through libraries such as **SpeechRecognition** and **PyAudio**.
- **Scientific computing:** Libraries like **NumPy**, **SciPy**, and **Pandas** provide advanced numerical computing capabilities for tasks like data analysis, machine learning, and more.
- **Education:** Python's easy-to-learn syntax and availability of many resources make it an ideal language for teaching programming to beginners.
- **Testing:** Python is used for writing automated tests, providing frameworks like **unit tests** and **pytest** that help write test cases and generate reports.
- **Gaming:** Python has libraries like **Pygame**, which provide a platform for developing games using Python.

- **IoT:** Python is used in IoT for developing scripts and applications for devices like **Raspberry Pi**, **Arduino**, and others.
- **Networking:** Python is used in networking for developing scripts and applications for network automation, monitoring, and management.
- **DevOps:** Python is widely used in DevOps for automation and scripting of infrastructure management, configuration management, and deployment processes.
- **Finance:** Python has libraries like **Pandas**, **Scikit-learn**, and **Statsmodels** for financial modeling and analysis.
- **Audio and Music:** Python has libraries like Pyaudio, which is used for audio processing, synthesis, and analysis, and Music21, which is used for music analysis and generation.
- **Writing scripts:** Python is used for writing utility scripts to automate tasks like file operations, **web scraping**, and **data processing**.

Uses of Python

It is used for:

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Versions of Python

- The most recent major version of Python is Python 3. However, Python 2, although not being updated with anything other than security updates, is still quite popular.

- Python can be written in a text editor. It is possible to write Python in an **Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.**

Python Basic Syntax

There is no use of curly braces or semicolons in Python programming language. It is an English-like language. Python uses indentation to define a block of code. Indentation is nothing but adding whitespace before the statement when it is needed.

For example -

1. `def func():`
2. `statement 1`
3. `statement 2`
4. `.....`
5. `.....`
6. `statement N`

In the above example, the statements that are the same level to the right belong to the function. Generally, we can use four whitespaces to define indentation.

Instead of Semicolon as used in other languages, Python ends its statements with a NewLine character.

Python is a case-sensitive language, which means that uppercase and lowercase letters are treated differently. For example, 'name' and 'Name' are two different variables in Python.

In Python, comments can be added using the '#' symbol. Any text written after the '#' symbol is considered a comment and is ignored by the interpreter. This trick is useful for adding notes to the code or temporarily disabling a code block. It also helps in understanding the code better by some other developers.

'If', 'otherwise', 'for', 'while', 'try', 'except', and 'finally' are a few reserved keywords in Python that cannot be used as variable names. These terms are used in the language for particular reasons and have fixed meanings. If you use these keywords, your code may include errors, or the interpreter may reject them as potential new Variables.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Example

```
print("Hello, World!")
```

Python Indentation

Indentation refers to the spaces at the beginning of a code line. In other programming languages, the indentation in code is for readability only, the indentation in Python is very important. Python uses indentation to indicate a block of code.

Example

```
if 5 > 2:  
    print("Five is greater than two!")
```

Python will give you an error if you skip the indentation:

Example

Syntax Error:

```
if 5 > 2:  
print("Five is greater than two!")
```

The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

Example

```
if 5 > 2:  
    print("Five is greater than two!")
```

```
if 5 > 2:
```

```
    print("Five is greater than two!")
```

You have to use the same number of spaces in the same block of code, otherwise Python will give you an error:

Example

Syntax Error:

```
if 5 > 2:
```

```
    print("Five is greater than two!")
```

```
    print("Five is greater than two!")
```

Python Variables

In Python, variables are created when you assign a value to it:

Example

Variables in Python:

```
x = 5
```

```
y = "Hello, World!"
```

Python has no command for declaring a variable.

You will learn more about variables in the Python Variables chapter.

Comments

Python has commenting capability for the purpose of in-code documentation.

Comments start with a #, and Python will render the rest of the line as a comment:

Example

Comments in Python:

```
#This is a comment.
```

```
print("Hello, World!")
```

Python Modules and Libraries

Real-world programs are complicated. Even simple software contains thousands of lines of code. Because of this, writing code in continuous flow is difficult for programmers and developers to grasp. Developers utilize modular programming to facilitate learning and make it logically separated. It is a method of breaking down huge coding tasks into shorter, more logical, and more

adaptable subtasks. Python's ease of use is one of its primary goals. Python has so many modules and libraries because of this.

Modules in Python

Modules are collections of related code that are packaged together in a Python program. Within a module, programmers can define functions, classes, or variables. It's also great to accommodate runnable codes within modules. They are, in other words, Python files that contain valid Python definitions and statements. When these files are created, the suffix.py is appended to them. By grouping related code into modules, the code becomes easier to understand and implement. It also organizes the code in a logical manner.

Modules are divided into two types.

- Predefined Modules(Built-in Modules)
- User-defined Modules

Predefined Modules: These are also referred to as built-in modules. Python has a huge number of built-in modules. Programmers can utilize these modules in Python applications by invoking their names along with the keyword 'import'. example Import math. The majority of the built-in Python modules are written in C and then combined with the Python interpreter to allow the interpreter to work alongside them. Examples: Math, datetime, statistics, random, os, sys, and other popular built-in Python modules.

Math: Gives users access to mathematical constants and pi and trigonometric functions.

Datetime: Provides classes for a simpler way of manipulating dates, times, and periods.

OS: Enables interaction with the base operating system, including administration of processes and file system activities.

Random: The random function offers tools for generating random integers and picking random items from a list.

JSON: JSON is a data structure that can be encoded and decoded and is frequently used in online APIs and data exchange. This module allows dealing with JSON.

Re: Supports regular expressions, a potent text-search and text-manipulation tool.

Collections: Provides alternative data structures such as sorted dictionaries, default dictionaries, and named tuples.

NumPy: NumPy is a core toolkit for scientific computing that supports numerical operations on arrays and matrices.

Pandas: It provides high-level data structures and operations for dealing with time series and other structured data types.

Requests: Offers a simple user interface for web APIs and performs HTTP requests.

User-defined Modules: Another advantage that Python gives its programmers is the ability to create user-defined modules. Python enables programmers to customize their own operations and gain control over them. Programmers can design their own collection of functions, variables, and classes within a user-defined module. To create a module just save the code you want in a file with the file extension .py:

External Modules: External modules must be downloaded from outside. They don't exist yet like the built-in ones. Installing them is a very simple task and can be done using the "pip install module_name" command in the compiler terminal. With so many modules available, familiarity with all seems to go a long way even for the best programmers. So you can search for modules to find and use them as needed.

Libraries

A library is an umbrella term that comprises a reusable set of Python code/instructions.

A Python library is typically a collection of similar modules grouped together under a single name. Developers commonly utilize it to share reusable code with the community. This eliminates the need to write Python code from scratch.

Developers and community researchers can construct their own set of useful functions in the same domain. When programmers and developers install the Python interpreter on their machines, standard libraries are included. Python libraries include matplotlib, Pygame, Pytorch, Requests, BeautifulSoup, and others.

- **Web development (Server-side) - Django Flask, Pyramid, CherryPy**
- **GUIs based applications - Tkinter, PyGTK, PyQt, PyJs, etc.**
- **Machine Learning - TensorFlow, PyTorch, Scikit-learn, Matplotlib, Scipy, etc.**
- **Mathematics - NumPy, Pandas, etc.**
- **BeautifulSoup:** a library for web scraping and parsing HTML and XML
- **Requests:** a library for making HTTP requests
- **SQLAlchemy:** a library for working with SQL databases

- **Kivy:** a framework for building multi-touch applications
- **Pygame:** a library for game development
- **Pytest:** a testing framework for Python Django
- **REST framework:** a toolkit for building RESTful APIs
- **FastAPI:** a modern, fast web framework for building APIs
- **Streamlit:** a library for building interactive web apps for machine learning and data science
- **NLTK:** a library for natural language processing

Difference Between Modules and Libraries

The following table shows the major differences between modules and libraries in python –

Modules	Libraries
A module is a set of code or functions with the.py extension.	A library is a collection of related modules or packages.
They are used by both programmers and developers.	Libraries are used by community members, developers and researchers.
The use of modules makes it easier to read the code.	Libraries provide no contribution for easy readability.
Modules are logical groups of functionality that programmers can import in order to reuse their code or set of statements.	Libraries enable users of programming languages, developers, and other researchers to reuse collections of logically related code.
When a Python programmer imports a module, the interpreter searches various locations for the module's definition or body.	Before we can use the libraries' modules or packages, we must first install them in our Python project. We generally use the pip install command.
Modules are generally written in Python with valid statements or codes.	Libraries, especially standard libraries, are usually developed in C or Python.

The basic goal of creating a module is to prevent DRY i.e, Don't Repeat Yourself.	Libraries do not have such a goal.
To return a sorted list of strings containing the function names specified within a module, we can use Python's built-in <code>dir()</code> function.	There is no explicit function that returns the number of modules in a library. Even so, programmers can utilize the <code>help()</code> function to get some information.
Popular built-in Python modules include os , sys , math , random , and so on.	Popular built-in Python libraries include Pygame , Pytorch , matplotlib , and more.

Python Basic Input and Output

Python Output: this is the process of displaying result on the screen.

In Python, we can simply use the `print()` function to print output. For example,

Print ('Python is powerful')

Output

Python is powerful

Here, the `print()` function displays the string enclosed inside the single quotation.

|

Syntax of print()

In the above code, the `print()` function is taking a single parameter. However, the actual syntax of the `print()` function accepts **5** parameters as follows:

Print (object=separator=end=file=flush=)

Here,

- **object** - value(s) to be printed
- **sep** (optional) - allows us to separate multiple **objects** inside `print()`.
- **end** (optional) - allows us to add add specific values like new line `"\n"`, tab `"\t"`
- **file** (optional) - where the values are printed. It's default value is `sys.stdout` (screen)
- **flush** (optional) - boolean specifying if the output is flushed or buffered. Default: `False`

Example 1: Python Print Statement

```
print ('Python Programming')  
print ('is very interesting')
```

Output

```
Python Programming  
is very interesting
```

In the above example, the `print()` statement only includes the **object** to be printed. Here, the value for **end** is not used. Hence, it takes the default value `"\n"`.

So we get the output in two different lines.

Example 2: Python print() with end Parameter

```
#print with end 'whitespace'  
Print ( 'Python programming', end= ' ' )  
Print ('is very interesting')
```

Output

```
Python programming is very interesting
```

Notice that we have included the `end= ' '` after the end of the first `print()` statement.

Hence, we get the output in a single line separated by space.

Example 3: Python print() with sep parameter

```
Print ('Next Year', 2025, 'See You on Top!', sep= '.')
```

Output

```
Next Year. 2025. See You on Top!
```

In the above example, the `print()` statement includes multiple **items** separated by a comma.

Notice that we have used the optional parameter `sep= "."` inside the `print()` statement.

Hence, the output includes items separated by `.` not comma.

Python Input: this is the process of entering data from the keyboard.

While programming, we might want to take the input from the user. In Python, we can use the `input()` function.

Syntax of input()

```
input(prompt)
```

Here, `prompt` is the string we wish to display on the screen. It is optional.

Example: Python User Input

```
#using input() to take user input
num = input ('Enter a number:')
print('You Entered:', num)
Print('Data type of num:', type(num))
```

Output

Enter a number: 10

You Entred: 10

data type of num:<class 'str'>

In the above example, we have used the `input()` function to take input from the user and stored the user input in the `num` variable .

It is important to note that the entered value **10** is a string, not a number. So, `type(num)` returns `<class 'str'>`.

To convert user input into a number we can use `int()` or `float()` functions as:

```
Num =int(input('Enter a number:'))
```

Here, the data type of the user input is converted from string to integer .

Example of Python Program

Write a program to compute and print the area of a rectangle (hint: area=length*width)

Solution

```
# Get input from the user
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))

# Calculate the area
```

```
area = length * width

# Display the result
print("The area of the rectangle is:", area)
```

Python Variables

Variables are simply containers for storing data values. They are memory locations whose contents can change during program execution. A variable can be seen as a placeholder for information you want Python to recall later in the coding process when you need to complete an action. Technically, a variable acts as an address where data is stored in memory. A python variable may be assigned a value of one type and then later re-assigned a value of a different type. Example: X= “orange” can later be X= 2.

Creating Variables

Unlike other languages like Java, Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

Example

```
x = 5
y = "John"
print(x)
print(y)
```

Variables do not need to be declared with any particular **type**, and can even change **type** after they have been set.

Example

```
x = 4    # x is of type int
x = "Sally" # x is now of type str
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

Example

```
x = str(3) # x will be '3'
y = int(3) # y will be 3
z = float(3) # z will be 3.0
```

Output Variables

The Python `print()` function is often used to output variables.

Example

```
x = "Python is awesome"
```

```
print(x)
```

In the `print()` function, you output multiple variables, separated by a comma:

Example

```
x = "Python"
```

```
y = "is"
```

```
z = "awesome"
```

```
print(x, y, z)
```

You can also use the `+` operator to output multiple variables:

Example

```
x = "Python "
```

```
y = "is "
```

```
z = "awesome"
```

```
print(x + y + z)
```

Python Variables - Naming Convention

Every Python variable should have a unique name like `a`, `b`, `c`. A variable name can be meaningful like `color`, `age`, `name` etc. There are certain rules which should be followed while naming a Python variable:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number or any special character like `$`, `(`, `*`, `%` etc.
- A variable name can only contain alpha-numeric characters and underscores (`A-z`, `0-9`, and `_`)
- Python variable names are case-sensitive which means `Name` and `NAME` are two different variables in Python.
- Python reserved keywords cannot be used in naming the variable.

If the name of variable contains multiple words, we should use these naming patterns –

- **Camel case** – First letter is a lowercase, but first letter of each subsequent word is in uppercase. For example: kmPerHour, pricePerLitre
- **Pascal case** – First letter of each word is in uppercase. For example: KmPerHour, PricePerLitre
- **Snake case** – Use single underscore (_) character to separate words. For example: km_per_hour, price_per_litre

|

Types of Python Variables

- Global Variables
- Local variables

Global Variables: Variables that are created outside of a function (as in all of the examples above) are known as global variables. Global variables can be used by everyone, both inside of functions and outside.

Example

Create a variable outside of a function, and use it inside the function

```
x=5
Y=10
def sum():
    Sum=x+y
    return sum
print(sum())
```

Output

15

Local variables

Python local variables are defined inside a function. We cannot access the variable outside the function.

Example

```
Def sum (x,y):  
    sum=x+y  
    return sum  
Print(sum(5,10))
```

Output

15

Python Constants

Python does not have any formally defined constants, however you can indicate a variable to be treated as constant by using all-caps names with underscores. For example, the name `PI_VALUE` indicates that you don't want the variable redefined or changed in any way. The naming convention using all-caps is sometimes referred to as **screaming snake case**- where the all-caps (screaming) and the underscores (snakes).

Python Data Types

Every value has a datatype, and variables can hold values. Python is a powerfully composed language; consequently, we don't have to declare the type of variable before using them. The interpreter binds the value implicitly to its type.

Example

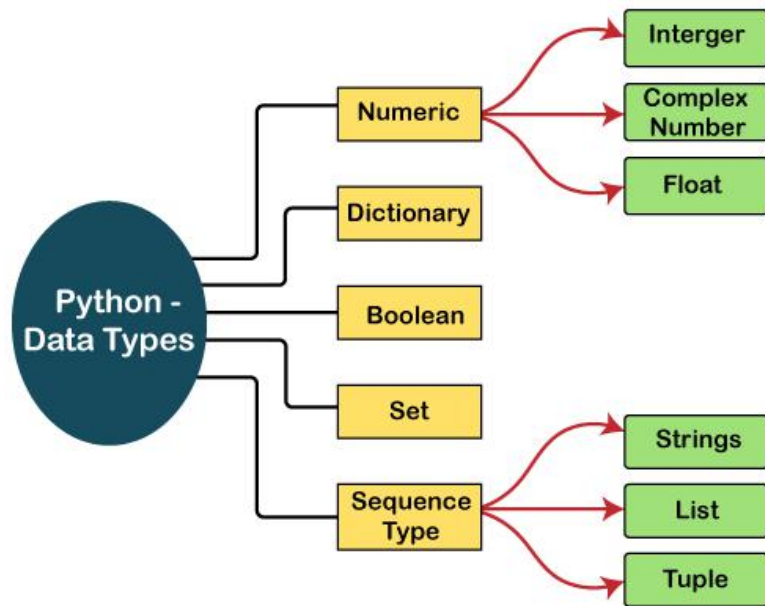
```
a = 5
```

We did not specify the type of the variable `a`, which has the value five from an integer. The Python interpreter will automatically interpret the variable as an integer.

Standard python data types

Python-defined data types include:

1. Numbers
2. Sequence Type
3. Boolean
4. Set
5. Dictionary



Numbers: Numeric values are stored in numbers. They include the whole number, float, and complex numbers.

- **Int:** These are whole numbers such as 10, 2, 29, - 20, - 150, and so on. An integer can be any length you want in Python.
- **Float:** Float stores decimal point numbers like 1.9, 9.902, 15.2, etc. It can be accurate to 15 decimal places.
- **Complex:** these numbers are made up of two parts- **real** and **imaginary**. They are separated by ' + ' or ' - ' signs. The imaginary part is suffixed by 'j' which is the imaginary number. Examples are 2.14j, 2.0 + 2.3j, etc.

Sequence Type: This include the following

String: The sequence of characters in the quotation marks can be used to describe the string. A string can be defined in Python using single, double, or triple quotes. When dealing with strings, the operation "hello"+" python" returns "hello python," and the operator + is used to combine two strings.

Lists: List in Python are like arrays in C, but lists can contain data of different types. The items of the list are separated with comma (,) and encased inside square bracket []. Example: [2024, "python", 3.22, 5+6j]

Tuple: a tuple is like a list. it also contains a collection of items from various data types. Unlike lists, tuples are enclosed within parenthesis ().

Dictionary: A dictionary is a key-value pair set arranged in any order. It stores a specific value for each key, like an associative array or a hash table. Value is any Python object, while the key can hold any primitive data type. The comma (,) and the curly braces are used to separate the items in the dictionary. Example: {1:'one', 2: 'two' 3: 'three'}

Boolean: True and False are the two default values for the Boolean type. Python bool() function allows you to evaluate the value of any expression and returns either True or False based on the expression. False can be represented by the 0 or the letter "F," while true can be represented by any value that is not zero.

Set: this is a collection but not an indexed or ordered collection as string, list or tuple. An object cannot appear more than once in a set unlike list and tuple where an object cannot appear more than once. The items of the set is enclosed within curly braces and separated by a comma or the built-in function set() is used to create the set. It can contain different kinds of values. Example: {2023, 3.11, 5+7j, }

Python Operators

Operators are special symbols that perform operations on variables and values. For example, Print(6+5)

Here, `+` is an operator that adds two numbers: 6 and 5.

Output: 11

Types of Python Operators

Different types of operators in Python include:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. Special Operators

1. Python Arithmetic Operators: Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Example

Sub = 10-5

Output: 5

Here, `-` is an arithmetic operator that subtracts two values or variables.

Operator	Operation	Example
<code>+</code>	Addition	<code>5 + 2 = 7</code>
<code>-</code>	Subtraction	<code>4 - 2 = 2</code>
<code>*</code>	Multiplication	<code>2 * 3 = 6</code>
<code>/</code>	Division	<code>10 / 3 = 3.3</code>
<code>//</code>	Floor Division	<code>10 // 3 = 3</code>
<code>%</code>	Modulo	<code>5 % 2 = 1</code>
<code>**</code>	Power	<code>4 ** 2 = 16</code>

2. Python Assignment Operators: Assignment operators are used to assign values to variables.

Example

```
# assign 5 to x
```

```
X = 5
```

Here, `=` is an assignment operator that assigns **5** to `x`.

The following are different assignment operators available in Python.

Operator	Name	Example
<code>=</code>	Assignment Operator	<code>a = 7</code>
<code>+=</code>	Addition	<code>a += 1 (a = a + 1)</code>

	Assignment	
<code>-=</code>	Subtraction Assignment	<code>a -= 3 (a = a - 3)</code>
<code>*=</code>	Multiplication Assignment	<code>a *= 4 (a = a * 4)</code>
<code>/=</code>	Division Assignment	<code>a /= 3 (a = a / 3)</code>
<code>%=</code>	Remainder Assignment	<code>a %= 10 (a = a % 10)</code>
<code>**=</code>	Exponent Assignment	<code>a **= 10 (a = a ** 10)</code>

3. Python Comparison Operators: Comparison operators compare two values/variables and return a boolean result: `True` or `False`. These operators are used in decision-making and loops.

For example,

`a = 5`

`b = 2`

Print (`a>b`)

Output: `True`

Here, the `>` comparison operator is used to compare whether `a` is greater than `b` or not.

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us <code>False</code>

<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us <code>True</code>
<code>></code>	Greater Than	<code>3 > 5</code> gives us <code>False</code>
<code><</code>	Less Than	<code>3 < 5</code> gives us <code>True</code>
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us <code>False</code>
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us <code>True</code>

4. Python Logical Operators: Logical operators are used to check whether an expression is `True` or `False`. They are used in decision-making.

Example

A=5

b=6

Print((a>2) and (b>=6))

Output: True

Here, `and` is the logical operator **AND**. Since both `a > 2` and `b >= 6` are `True`, the result is `True`.

Operator	Example	Meaning
<code>and</code>	a and b	Logical AND: <code>True</code> only if both the operands are <code>True</code>
<code>or</code>	a or b	Logical OR: <code>True</code> if at least one of the operands is <code>True</code>
<code>not</code>	not a	Logical NOT: <code>True</code> if the operand is <code>False</code> and vice-versa.

5. Python Bitwise operators: Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

Example, 2 is 10 in binary, and 7 is 111.

In the table below: Let $x = 10$ (0000 1010 in binary) and $y = 4$ (0000 0100 in binary)

Operator	Meaning	Example
&	Bitwise AND	$x \& y = 0$ (0000 0000)
	Bitwise OR	$x y = 14$ (0000 1110)
~	Bitwise NOT	$\sim x = -10$ (1111 0101)
^	Bitwise XOR	$x \wedge y = 14$ (0000 1110)
>>	Bitwise right shift	$x \gg 2 = 2$ (0000 0010)
<<	Bitwise left shift	$x \ll 2 = 40$ (0010 1000)

6. Python Special operators: Python language offers some special types of operators like the identity operator and the membership operator. They are described below with examples.

Identity operators: In Python, `is` and `is not` are used to check if two values are located at the same memory location. It is important to note that having two variables with equal values doesn't necessarily mean they are identical.

Operator	Meaning	Example
<code>is</code>	True if the operands are identical (refer to the same object)	<code>x is True</code>
<code>is not</code>	True if the operands are not identical (do not refer to the same object)	<code>x is not True</code>

Example 4: Identity operators in Python

```

x1=5
y1=5
x2 = 'Hello'
y2 = 'Hello'
X3= [1,2,3]
Y3= [1,2,3]
Print(x1 is not y1) # prints False
Print (x2 is y2)   #prints True
Print (x3 is y3)   #prints False

```

Here, we see that `x1` and `y1` are integers of the same values, so they are equal as well as identical. The same is the case with `x2` and `y2` (strings). But `x3` and `y3` are lists. They are equal but not identical. It is because the interpreter locates them separately in memory, although they are equal.

Membership operators: In Python, `in` and `not in` are the membership operators. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

In a dictionary, we can only test for the presence of a key, not the value.

Operator	Meaning	Example
<code>in</code>	<code>True</code> if value/variable is found in the sequence	<code>5 in x</code>
<code>not in</code>	<code>True</code> if value/variable is not found in the sequence	<code>5 not in x</code>

Example 5: Membership operators in Python

```

message = 'Hello world'
dict1={1:'a'. 2:'b'}
#check id 'H' is present in string
print ('H' in message) #print True
#check if 'hello' is present in message string
print ('hello' is not in message) #prints True
#check if '1' key is present in dict1
print (1 in dict1) #prints True
#check if 'a' key is present in dict1
print ('a' in dict1) #prints False

```

Output

True

True

True

False

Here, 'H' is in message, but 'hello' is not present in message (remember, Python is case-sensitive).

Similarly, 1 is key, and 'a' is the value in dictionary dict1. Hence, 'a' in dict1 returns False.

Python Control Structures

A program's control flow is the order in which the program's code executes. The control flow of a Python program is regulated by conditional statements, loops and function calls.

Python has three types of control structures:

- **Sequential** - default mode
- **Selection** - used for decisions and branching
- **Repetition** - used for looping, i.e., repeating a piece of code multiple times.

1. Sequential: Sequential statements are a set of statements whose execution process happens in a sequence. The problem with sequential statements is that if the logic has broken in any one of the lines, then the complete source code execution will break.

2. Selection/Decision control statements

In Python, the selection statements are also known as Decision control statements or branching statements. The selection statement allows a program to test several conditions and execute instructions based on which condition is true.

Some decision control statements are:

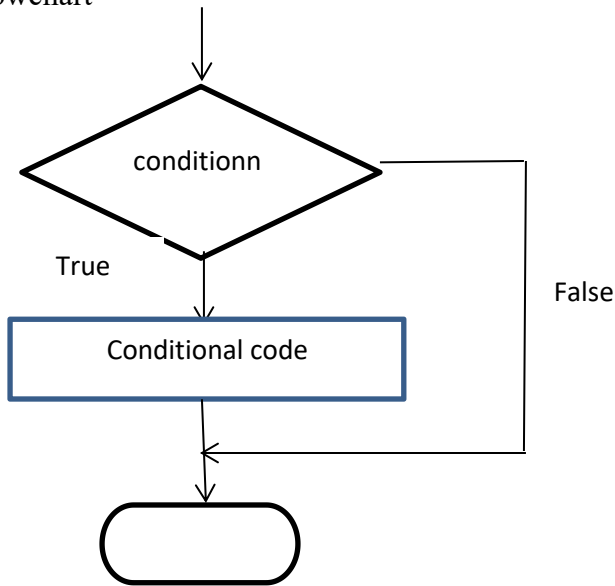
- `if`
- `if-else`
- `nested if`
- `if-elif-else`

If Statement: this allows a particular code to be executed when a certain condition is true or satisfied. An `if` statement tests for only one condition.

Syntax

1. `if` <conditional expression> :
2. The code block to be executed `if` the condition `is` True

Flowchart



Example

Score =75

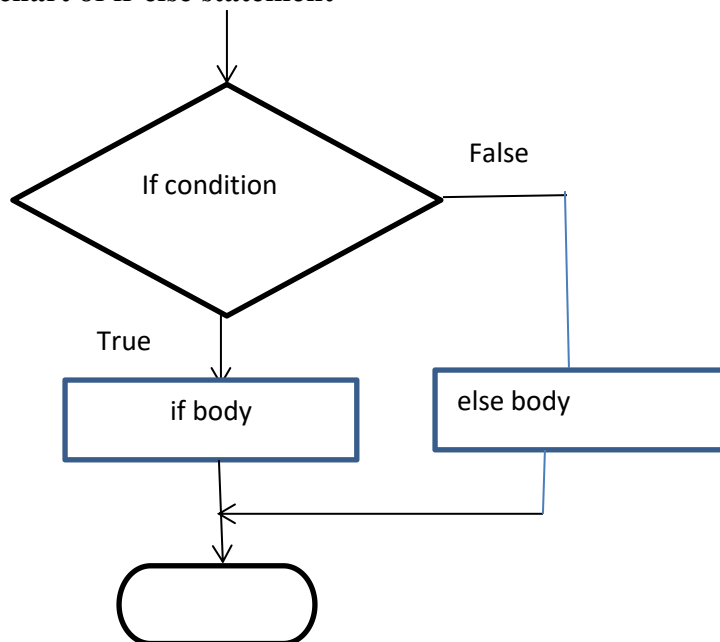
If score >= 60:

Print (“congratulations! You passed.”)

Here, the code inside the ‘if’ block will be executed only if the ‘score’ is greater than 60. since the ‘score’ is 75, the message “congratulations! You passed will be displayed on the screen

if-else Statement: The if-else statement evaluates the condition and will execute the body of ‘if’ if the test condition is True, but if the condition is False, then the body of ‘else’ is executed.

Flowchart of if-else statement



Example

score=45

If score >= 60:

Print("congratulations! You passed")

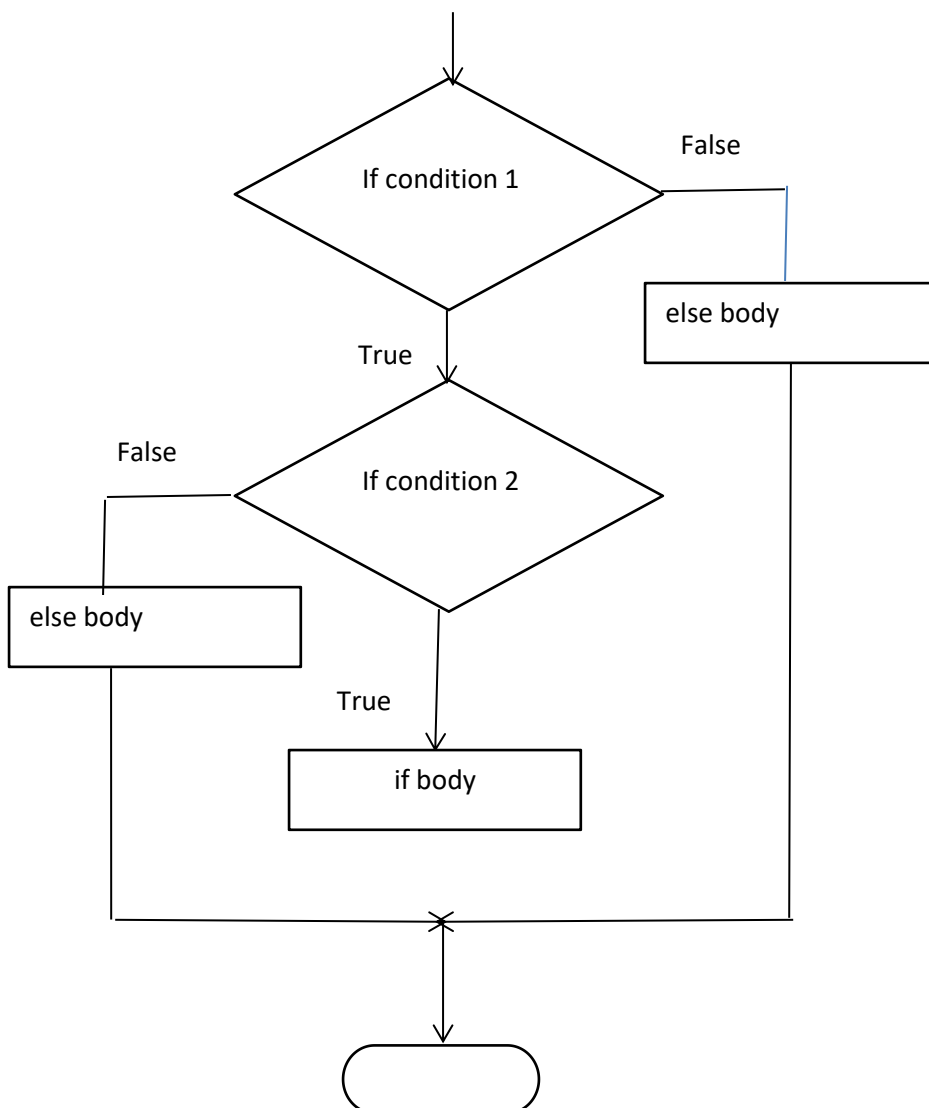
else:

Print("sorry try again")

Here, if the 'score' is less than 60, the code inside the 'else' block will be executed.

Nested if statement: Nested if statements are an if statement inside another if statement. It is used when you need to check for multiple conditions within another condition.

Flowchart of nested if statement



Example

Name= "Faith"

Score= 79

"If name== "Faith":

 If score>=80:

 Print("congratulations! You got A".)

 else:

 Print("good effort Faith, keep it up.")

Else:

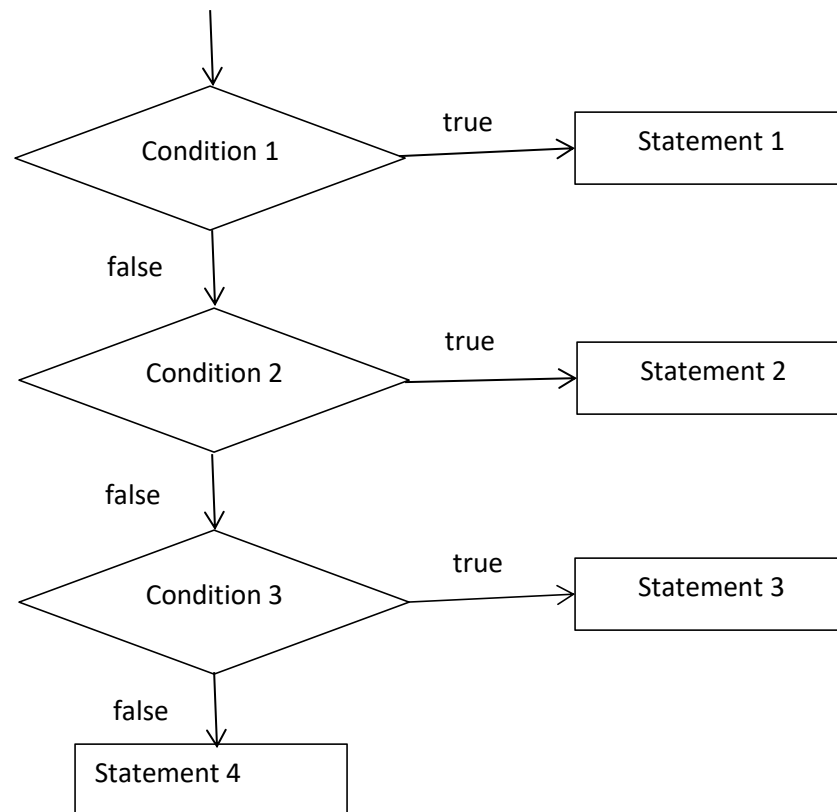
 Print("you are doing well but this message is for Faith.")

Output: "good effort Faith, keep it up.

In the example above, the program first checks if 'name' is "Faith". if true, it checks the 'score'. since it's 79, the inner 'else' block is executed, printing "good effort Faith, keep it up").

if-elif-else Statement: The if-elif-else statement is used when you have multiple conditions to check. It allows you to test multiple conditions and execute the corresponding block of code for the first true condition encountered. If the condition in the 'if' and all 'elif' statements evaluate to false, the 'else' block is executed.

Flowchart of If-elif-else



Example

Score= 84

If score >= 90

Print("excellent! You got an A.")

Elif score >= 80:

Print("nice one! You got a B.")

Elif score >= 70:

Print("you got C.")

Else:

Print("you need to improve. You got F.")

In the example above, the program checks the 'score' against multiple conditions. The code inside the first true condition's block will be executed . since the 'score' is 84.

Output

nice one!you got a B.

(

3. Repetition

A repetition statement is used to repeat a group(block) of programming instructions.

In Python, we generally have two loops/repetitive statements:

- for loop
- while loop

for loop: A for loop is used to iterate over a sequence that is either a list, tuple, dictionary, or a set.

We can execute a set of statements once for each item in a list, tuple, or dictionary.

Example

```
student_names = ["Alice", "Bob", "Charlie", "David"]
```

```
for name in student_names:  
    print("Student:", name)
```

outputs:

Student: Alice

Student: Bob

Student: Charlie

Student: David

while loop

In Python, while loops are used to execute a block of statements repeatedly until a given condition is satisfied. Then, the expression is checked again and, if it is still true, the body is executed again. This continues until the expression becomes false

Example

The following code iterates from 0 to 4 and prints each value using a while loop. It prints End to signify the end of the program after the loop is completed.

```
m= 5
```

```
i= 0
```

```
While i < m:
```

```
    Print(i, end= " ")
```

```
i= i+ 1
```

```
Print("End")
```